

Overview

LLM quality in production is not a one-time training outcome. It is a continuous loop of: 1. evaluate, 2. diagnose failure patterns, 3. collect better data, 4. retrain or retune, 5. redeploy with guardrails.

This chapter covers evaluation systems, data flywheel design, and operational controls for continuous post-training.

Evaluation Taxonomy

Offline evaluation

Runs on static benchmark sets before deployment.

Useful for: - candidate comparison, - regression checks, - release gate decisions.

Online evaluation

Runs against real traffic signals.

Useful for: - behavior drift detection, - business metric tracking, - controlled rollout validation.

Human evaluation

Critical when automatic metrics under-specify quality.

Useful for: - nuanced instruction quality, - policy adherence review, - domain expert verification.

What to Measure

Quality metrics

- task success rate,
- groundedness/faithfulness,
- schema adherence,
- refusal correctness.

Safety metrics

- harmful output rate,
- policy violation categories,
- prompt injection success rate.

Reliability metrics

- latency percentiles,
- timeout/error rates,
- degraded mode activation frequency.

Business metrics

- user satisfaction,
- conversion/task completion,
- escalation rate,
- cost per successful task.

Data Flywheel Design

A production data flywheel should turn failures into training assets.

Core loop

1. Collect prompts, outputs, traces, and feedback.
2. Identify high-impact failure clusters.
3. Prioritize clusters by severity and frequency.
4. Generate or label corrective examples.
5. Retrain/retune candidate model.
6. Re-run regression and safety suites.
7. Promote only if gate thresholds pass.

Data governance controls

- remove sensitive/PII content where required,
- version datasets and labeling policies,
- keep reviewer notes for auditability,
- track provenance for each training sample group.

Continuous Post-Training Operations

Release gates

A robust gate includes: - baseline comparison delta, - must-pass safety constraints, - no critical regressions on high-value slices, - cost/latency budget compliance.

Canary and rollback

Deploy tuned candidates gradually: - shadow mode first, - small canary percentage, - live metric comparison with control, - automatic rollback triggers for severe regression.

Incident handling

When quality incidents occur: 1. classify failure type (retrieval, policy, model behavior, integration), 2. isolate blast radius, 3. apply mitigation (routing rollback, stricter guardrails, temporary model swap), 4. feed incident samples into next training cycle.

Evaluation for RAG and Agents

For RAG/agent systems, split evaluation by component: - retrieval quality, - planning/tool-use correctness, - final response quality, - safety and policy adherence.

Only scoring final answer quality can hide retrieval or orchestration defects.

Production Case Studies & Exceptions

Case 1: Finance report assistant

Issue: high lexical quality but factual grounding errors. Fix pattern: groundedness gate + citation validation + retrieval refresh policy. Exception: strict citation enforcement can reduce response fluency unless prompt templates are adjusted.

Case 2: Customer support co-pilot

Issue: rising escalation rate after new adapter release. Fix pattern: automatic rollback + slice-based error analysis + targeted retraining data. Exception: if failure is due to upstream API schema drift, retraining is not the first fix.

Case 3: Internal engineering assistant

Issue: strong offline benchmarks but weak real usage satisfaction. Fix pattern: online eval program with user feedback taxonomy and weekly review cadence. Exception: feedback can be noisy; combine qualitative review with quantitative thresholds.

Interview Questions & Answers

1. **Q:** Why is offline eval alone insufficient?
A: Real traffic distribution and user behavior differ from static test sets.
2. **Q:** What is a data flywheel in LLMops?
A: A loop that converts production failures and feedback into better training data and improved model versions.
3. **Q:** First release gate you would define?
A: No regression on critical task slices versus baseline.
4. **Q:** Why include safety gates separately from quality gates?
A: High task quality can still violate safety policy.
5. **Q:** What is a rollback trigger example?
A: Safety violation spike above threshold or large drop in task success.
6. **Q:** How do you evaluate agent systems?
A: Evaluate retrieval, planning/tool execution, and final output separately.
7. **Q:** Why version evaluation datasets?
A: To keep comparisons reproducible across releases.
8. **Q:** Human eval vs automatic metrics?
A: Automatic scales fast; human eval captures nuance and policy edge cases.
9. **Q:** Common anti-pattern in continuous tuning?
A: Training frequently without stable regression benchmarks.
10. **Q:** Why track cost per successful task?
A: It links quality and economics for product viability.
11. **Q:** What does slice-based evaluation mean?
A: Measuring performance on specific cohorts or failure-prone categories.
12. **Q:** Why keep prompt/output traces?
A: They are essential for root-cause analysis and dataset curation.
13. **Q:** What is canary release in LLMops?
A: Serving a new model to a small traffic share before full rollout.
14. **Q:** How do you avoid noisy feedback loops?
A: Use taxonomy-guided labeling and confidence-weighted sampling.
15. **Q:** Why compare against baseline every cycle?
A: To ensure changes are real improvements, not random variation.
16. **Q:** What is groundedness?
A: Degree to which response claims are supported by trusted sources/context.
17. **Q:** Which incidents should trigger retraining vs infra fix?
A: Behavior/policy errors suggest retraining; schema/connectivity errors suggest infra fixes.
18. **Q:** Why separate control and treatment during rollout?
A: To attribute metric changes to the model release.
19. **Q:** What must be in an eval report?
A: metric deltas, slice regressions, safety outcomes, and go/no-go decision rationale.

20. **Q:** One sentence production guidance?

A: Treat evaluation as a productized system, not a one-off experiment.

References

- OpenAI fine-tuning guide: <https://platform.openai.com/docs/guides/supervised-fine-tuning>
- DeepLearning.AI LLMOps course: <https://www.deeplearning.ai/courses/llmops>
- TRL docs: <https://huggingface.co/docs/trl/main/index>
- MCP architecture for observability context: <https://modelcontextprotocol.io/docs/learn/architecture>